WELLS FARGO

Privacy-Preserving Federated System for Adaptive Malicious Prompt Mitigation in LLMs Using Behavioral Signatures, Trust-Weighted Aggregation, and Auditable Rollback Control

D Rahul Prasanth

© 2020 Wells Fargo Bank, N.A. All rights reserved. For public use.

Problem Statement

As large language models (LLMs) such as GPT are increasingly used in sensitive, user-facing applications, a critical and unsolved challenge remains, How can LLM providers continue to improve their ability to detect and mitigate malicious intrusions (e.g., jailbreaks, social engineering, code injections) without centralizing user data and preserving privacy?

In particular, current methods face several key limitations:

- Centralization risks: Traditional approaches rely on consolidating user data or model updates on a central server, which could expose sensitive or private information.
- Uniform security mechanisms: Existing models use a single set of security classifiers that often ignore user or organization-specific risk tolerances, behavioral patterns, or threat profiles.
- Lack of adaptive control: Once harmful or aberrant model behaviors are discovered, current systems lack accurate, privacy-respecting remediation or isolation mechanisms to effectively contain and remediate threats, and often require laborious manual intervention.
- Insufficient auditability: The ability to cryptographically audit or replay the effect of local model updates on global security is limited, especially without sharing or exposing raw user data.

Needs to solve the problem

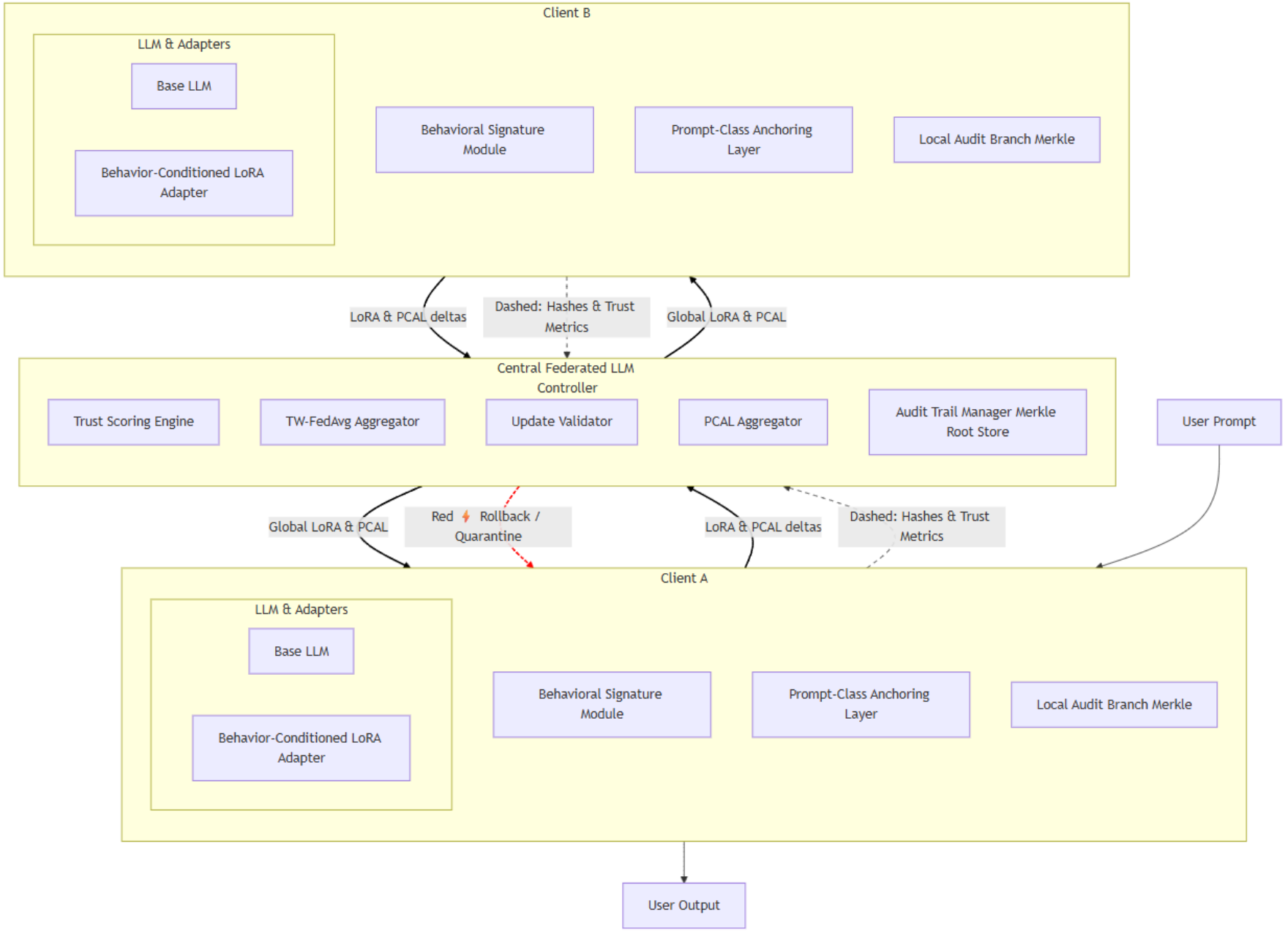
There is a need for a federated, privacy-preserving learning framework that enables:

- Continuous, context-aware improvements in the immediate detection and mitigation of malicious activity
- Personalized, adaptive security mechanisms that reflect user or organization-specific behavior
- Robust, autonomous refactoring and isolation to protect against model drift or negative updates
- Cryptographically secure, privacy-preserving auditing and source tracking of updates and interventions.

Key Claims:

- Federated LLM adaptation through behavioral signature-conditioned, parameter-efficient adapters.
- Trust-Weighted Federated Averaging (TW-FedAvg) for aggregation, modulating client influence via reputation metrics (gradient drift, anomaly rate, diversity).
- Prompt-Class Anchoring Layer (PCAL) for distributed, privacy-preserving classification of prompt intent/risk, federated independently of base LLM.
- Autonomous quarantine and rollback protocol for clients whose updates deviate or show risk signatures, with cryptographic audit trail

Architecture:



Steps:

1. User Interaction at Client Node

Step 1: User submits a prompt to their local client application/interface.

Step 2: The client routes the prompt to the on-device LLM, which consists of:

- The base LLM (e.g., GPT variant)
- LoRA adapters conditioned by the client's behavioral signature profile
- The Prompt-Class Anchoring Layer (PCAL) for prompt safety classification

2. Local Inference & Safety Check

Step 3: The LLM processes the prompt.

Step 4: The PCAL head classifies the prompt in real time (e.g., Safe, Jailbreak, Social Engineering).

- If the prompt is safe, the adapter generates a standard completion.
- If risky/malicious, the client may suppress, rephrase, or warn the user, applying local mitigation.

3. Local Update and Training Cycle

Step 5: The client logs prompt-class outcomes, anomalies, and behavioral data locally, updating:

- LoRA adapters: Fine-tuned with user-specific context and behavioral signatures
- PCAL head: Locally updated with new examples, reinforcing prompt classification performance

Step 6: Client computes weight deltas for LoRA adapters and the PCAL head, tags them with a behavioral context hash.

Steps:

4. Privacy-Preserving Synchronization

Step 7: At set intervals or based on a local trigger, the client prepares an update package:

- Contains LoRA and PCAL delta weights (no raw data or prompts)
- Includes the behavioral signature hash, trust metrics, and cryptographic hashes (Merkle branches) for update lineage

Step 8: Client transmits only these privacy-preserving updates/assets to the central federated server.

5. Federated Aggregation & Trust Modulation

Step 9: The central server:

- Receives updates from all participating clients
- Validates quality and trust features (gradient drift analysis, prompt-class anomaly rates, reputational signals)
- Down-weights or ignores suspicious or low-trust updates
- Aggregates trustworthy updates using Trust-Weighted Federated Averaging (TW-FedAvg)
- Aggregates only PCAL head weights, not base model or raw data

6. Rollback & Quarantine Control

Step 10: If a client's update diverges excessively, seems adversarial, or fails prompt-class validation:

- The server flags the client
- Instructs the client to rollback to a previous adapter and PCAL version (client initiates the rollback locally)
- Updates the audit log with cryptographic evidence (Merkle proof) for traceability

Step 11: Quarantined clients are isolated in the next federation round until remediation

Steps:

7. Auditable History and Forensic Replay

Step 12: All client/server update transactions are hashed and merged into a distributed Merkle audit chain:

- Clients retain local branches (with version lineage)
- Server keeps only the hashes/Merkle roots—no raw data or prompts
- This enables privacy-preserving forensic audit and replay if ever required, supporting regulatory and internal compliance

8. Update Distribution and Continuous Learning

Step 13: Once aggregation is done, the central server sends back the improved global adapter weights (LoRA) and updated PCAL head weights to all clients.

Step 14: Clients receive these updates, integrate the new weights, and start a new inference/fine-tuning cycle.



Thank you